The **Project Sentinel** concept is compelling — a tamper-proof, multi-modal "black box" for operating rooms could genuinely advance patient safety, surgical training, and medico-legal fairness. The provided execution plan is solid for a hackathon/demo prototype: focused, pragmatic, and staged correctly (core sync first, polish last).

I'll give structured feedback, refinements, implementation tips, and concrete next steps based on the requirements and folder structure. This assumes you're building in Python with the listed tools (OpenCV, Streamlit, etc.) for a realistic proof-of-concept.

1. Video Source Recommendations

You need a visually dynamic laparoscopic video (preferably cholecystectomy) that's free/public for demo use. Avoid copyrighted YouTube clips if possible — many hospital/educational channels have restrictions.

Top practical options (from public datasets and sources):

- **Cholec80 / CholecT50 / Endoscapes** (strongly recommended):
  - These are the gold-standard public laparoscopic cholecystectomy datasets from the CAMMA group (University of Strasbourg).
  - Cholec80: 80 full videos → download from http://camma.u-strasbg.fr/datasets (registration usually required, but academic/public use is free).
  - Pick 1–2 longer videos (they're typically 20–60 minutes).
  - Why ideal: Real surgical footage, dynamic tool movement, occasional alarms/motion changes → perfect for anomaly demo.
  - License: Usually research/educational (CC-BY-NC-SA or similar) — fine for hackathon.
- **CholecSeg8k** (Kaggle): https://www.kaggle.com/datasets/newslab/cholecseg8k
  - Extracted frames + some video clips from Cholec80.
  - Quick to download if you just need short segments.
- Fallback (quick & dirty): Search YouTube for "laparoscopic cholecystectomy full surgery unedited" from reputable channels like laparoscopyhospital.com or SAGES — many offer full HD videos. Download one ~10–30 min clip (use yt-dlp if allowed in your environment). Overlay "OR-CAM-01 | LIVE" as planned.

Place it in data/surgery.mp4 (or trim to 3–5 min for faster testing using ffmpeg or OpenCV).

2. Key Implementation Guidance by Module

backend/simulator/vitals_generator.py

- Use 10 Hz sampling → every 0.1 s.
- Realistic signals:

Python

```
import numpy as np
```

```python
import time
import csv
from datetime import datetime

class VitalsGenerator:
    def __init__(self, session_dir):
        self.t = 0
        self.hr_base = 75
        self.spo2_base = 98
        self.bp_sys_base = 120
        self.bp_dia_base = 80
        self.session_dir = session_dir
        self.csv_path = f"{session_dir}/vitals.csv"
        with open(self.csv_path, 'w', newline='') as f:
            writer = csv.writer(f)
            writer.writerow(["timestamp", "HR", "SpO2", "BP_sys", "BP_dia"])

    def get_vitals(self):
        noise = np.random.normal(0, 2)
        resp_mod = 5 * np.sin(0.25 * self.t)  # respiratory variation

        hr = self.hr_base + resp_mod + noise
        spo2 = self.spo2_base + np.random.normal(0, 0.5)
        bp_sys = self.bp_sys_base + resp_mod * 2 + noise * 3
        bp_dia = self.bp_dia_base + noise * 2

        # Inject anomalies (example timeline in seconds)
        if 40 < self.t < 50:
```

```
            hr = 145 + noise * 5          # tachycardia
        elif 80 < self.t < 90:
            spo2 = 82 + noise             # hypoxia
            hr += 20
        elif 120 < self.t < 130:
            hr = 110                      # partial recovery

        ts = datetime.now().isoformat()
        with open(self.csv_path, 'a', newline='') as f:
            writer = csv.writer(f)
            writer.writerow([ts, round(hr), round(spo2), round(bp_sys), round(bp_dia)])

        self.t += 0.1
        return {"timestamp": ts, "HR": hr, "SpO2": spo2, "BP_sys": bp_sys, "BP_dia": bp_dia}
```

- Run in a thread, sleep(0.1).

backend/simulator/video_stream.py

- Use OpenCV:

Python

```
import cv2
import time

cap = cv2.VideoCapture("data/surgery.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break
```

```
    # Overlay
    cv2.putText(frame, "OR-CAM-01 | LIVE", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)


    # yield frame + clock.now() timestamp
    time.sleep(1 / fps)
```

backend/sync/global_clock.py & stream_aligner.py

- Simple monotonic clock:

Python

```
import time


class ORClock:
    def __init__(self):
        self.start = time.time()


    def now(self):
        return time.time() - self.start
```

- Aligner: Collect latest frame + vitals sample at ~same clock.now() → store paired entries (e.g., list of dicts or append to HDF5/CSV).

backend/ai/anomaly_rules.py & motion_detector.py

- Rules-based (fast & reliable for demo):

Python

```
def detect_anomaly(vitals):
    events = []
    if vitals["HR"] > 130:
        events.append({"time": clock.now(), "type": "Tachycardia", "severity": "high"})
    if vitals["SpO2"] < 85:
```

```
        events.append({"time": clock.now(), "type": "Hypoxia", "severity": "critical"})
    # add BP <90/50 etc.
    return events
```

- Motion (simple frame diff):

Python

```
prev_gray = None
def motion_detect(frame):
    global prev_gray
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    if prev_gray is not None:
        diff = cv2.absdiff(prev_gray, gray)
        mean_diff = diff.mean()
        if mean_diff > 15:  # tune threshold
            return {"type": "Erratic Motion", "severity": "medium"}
    prev_gray = gray
    return None
```

- Collect events → save to events.json.

backend/integrity/merkle.py

- Simple chunked hashing (every 5 s):

Python

```
import hashlib
import json

hashes = []

def update_hash(video_chunk_bytes, vitals_str):
    combined = video_chunk_bytes + vitals_str.encode()
```

```
    h = hashlib.sha256(combined).hexdigest()

    hashes.append(h)

    # In real: append to Merkle tree or log to file

    with open("hashes.json", 'w') as f:

        json.dump({"hashes": hashes, "root": compute_merkle_root(hashes)}, f)


def verify_integrity():

    # recompute all → compare root

    # for demo: manually edit mp4 frame → show mismatch
```

- For true Merkle: use a library like merkle-tools or simple pairwise.

frontend/app.py (Streamlit)

- Use st.video() for playback.
- Plotly line chart for vitals (real-time update with st.empty() + thread).
- st.slider for scrub → jump to time (map to video seconds).
- Overlay pins: Use Plotly annotations or custom timeline component.
- Integrity: Button "Verify Integrity" → run check → show green/red badge.

3. Demo Polish & "Wow" Moments

- Start stable → inject tachycardia (alarm sound + red flash + pin).
- Scrub to event → vitals + video aligned.
- Edit one frame in surgery.mp4 (use ffmpeg or editor) → verify → 🔴 **TAMPER DETECTED** banner.
- End pitch: "In aviation, the black box tells the truth. In surgery, Project Sentinel does the same — immutable, synchronized, AI-assisted."

4. Quick Wins & Risks

- **Priority 1**: Get video playing + vitals graphing + basic sync (same timeline) — do this in first 6–8 hours.
- Risk: Streamlit video seek can be laggy — pre-index frames or use short clip.
- No blockchain needed — filesystem + SHA256 chain/Merkle root is enough for demo.
- Add audio: Play beep.wav on anomaly (use st.audio or playsound).

If you hit specific blockers (e.g., video download link issues, code errors, or need sample code for Merkle), share details — I can drill deeper.

Good luck building Sentinel — nail the core sync early, and you'll have a standout demo. This could actually matter beyond the hackathon. 🚀

🧠 PROJECT SENTINEL — FULL EXECUTION PLAN

🎯 Final Deliverable

A web app that:

- Plays simulated surgery video
- Shows synchronized vitals
- Auto-detects anomalies
- Shows event pins
- Proves tamper detection

📁 FINAL FOLDER STRUCTURE

Do NOT improvise. Use this.

```
sentinel/
│
├── backend/
│   ├── simulator/
│   │   ├── video_stream.py
│   │   ├── vitals_generator.py
│   │   ├── alarm_engine.py
│   │
│   ├── sync/
│   │   ├── global_clock.py
│   │   ├── stream_aligner.py
│   │
│   ├── ai/
│   │   ├── anomaly_rules.py
```

```
│   │   ├── motion_detector.py
│   │
│   ├── integrity/
│   │   ├── merkle.py
│   │   ├── hash_logger.py
│   │
│   ├── storage/
│   │   ├── session_manager.py
│   │
│   └── main.py
│
├── frontend/
│   ├── app.py  (Streamlit)
│
├── data/
│   ├── surgery.mp4
│
└── requirements.txt
```

This structure alone shows **engineering maturity**.

❤️ VITALS SIMULATOR (DETAILED DESIGN)

Goal:

Generate ICU-like waveform + anomalies.

Base Signal Model

Use:

signal = baseline + sin_wave + random_noise

Example HR:

- baseline = 75
- sin_wave → respiration effect
- noise → measurement variation

Inject Events

Hardcode timeline:

t = 40 sec → tachycardia spike
t = 80 sec → hypoxia
t = 120 sec → recovery

Example logic:

```
if 40 < t < 50:
  HR = 140 + noise
```

This creates **dramatic demo moments.**

Store output:

timestamp, HR, SpO2, BP

Sampling rate:

👉 10 Hz (realistic + light)

📹 VIDEO STREAM SIMULATOR

Use OpenCV.

Core logic:

- read frame
- attach timestamp
- send to sync engine

Simulate real-time:

sleep(1 / fps)

Hackathon trick:

Add overlay:

"OR-CAM-01 | LIVE"

Judges psychologically believe.

⏱️ GLOBAL CLOCK DESIGN

This is core innovation.

Create single source:

```
class ORClock:
  start_time
  get_current_time()
```

Every stream must call:

```
clock.now()
```

Now you can align:

- video frame
- vital sample
- alarm event

This is what makes project **non-toy.**

🔔 ALARM ENGINE

Simple rule:

If anomaly detected:

- play beep.wav
- push alert event

Also show UI flash.

Multi-sensory demo → very powerful.

🤖 AI EVENT TAGGING

Keep simple.

Rule Engine

if HR > 130 → Tachycardia
if SpO2 < 85 → Hypoxia
if BP < threshold → Hypotension

Motion Detection

Use:

```
frame_diff = abs(prev - current)
if mean_diff > threshold → Erratic Motion
```

Create event object:

```
{
  time,
  type,
  severity
}
```

Store in JSON.

🔐 MERKLE INTEGRITY SYSTEM

Chunk strategy:

Every 5 seconds:

hash = SHA256(video_chunk + vital_chunk)

Append to list.

Final:

root_hash

Tamper demo:

- edit frame manually

- recompute → mismatch

Show:

🔴 TAMPER DETECTED

This is **final WOW moment.**

🗄️ STORAGE DESIGN

Session folder:

```
session_01/
  vitals.csv
  events.json
  hashes.json
  video_chunks/
```

No database needed.

Hackathon rule:

👉 filesystem is enough.

🖥️ UI DESIGN (STREAMLIT)

Layout:

```
---------------------------------
| Video Player                  |
---------------------------------
| Vital Graph (real time)       |
---------------------------------
| Timeline with event markers   |
---------------------------------
| Integrity Status              |
---------------------------------
```

Features:

- slider scrub timeline

- click event → jump video
- show anomaly description

Use:

- plotly for vitals
- streamlit video player

🎬 DEMO SCRIPT (PRACTICE THIS)

You say:

1️⃣ Surgery begins
2️⃣ Vitals stable
3️⃣ Suddenly tachycardia
4️⃣ Alarm rings
5️⃣ AI drops event
6️⃣ Scrub timeline
7️⃣ Show synchronized playback
8️⃣ Tamper video
9️⃣ Integrity fails

End line:

> “This ensures medico-legal truth cannot be altered.”

Judges silent → then clap.

🎥 SURGERY VIDEO SOURCES

Search:

- laparoscopic surgery public dataset
- endoscopic video dataset
- YouTube laparoscopic footage

Good keywords:

- cholecystectomy laparoscopy
- endoscopic surgery dataset

Pick:

👉 visually dynamic video

Not static.

⏳ BUILD TIMELINE (REALISTIC)

If 36-hour hackathon:

Phase 1 (6h)

Vitals + video simulator

Phase 2 (6h)

Global sync

Phase 3 (6h)

UI basic player

Phase 4 (6h)

AI tagging

Phase 5 (4h)

Merkle integrity

Phase 6 (4h)

Demo polish

Phase 7 (4h)

Pitch practice

⚠️ CRITICAL STRATEGY RULE

You must reach:

👉 “Basic sync + playback working” early

Everything else optional.

Most teams fail because:

They build features before core works.